Login with Everything

Andrew Miller, Xinyuan Sun, Novel Tokens
Teleport / Flashbots[X]
socrates1024@gmail.com

March 2026

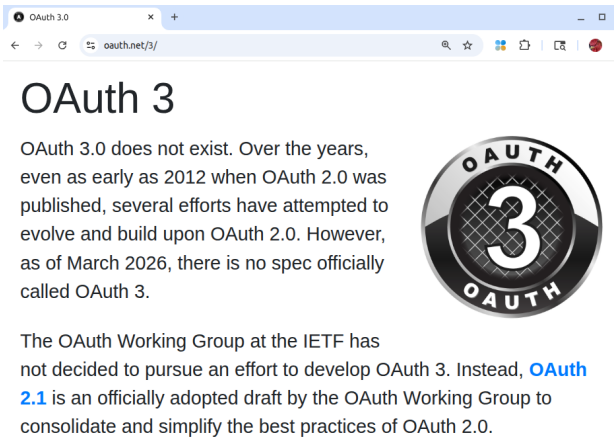

Figure 1: OAuth 3 does not exist. [**?**]

Abstract

Tired of boring credentials? Every website on the internet can be an identity provider if you're determined enough. Actually all you need is to run a browser in an attested environment. This is easy enough with confidential compute, or even just Github actions... and a VPN. As an application, we welcome you to a message board where you can log in with everything

1 Introduction

If you want to know whether someone has a GitHub account, why not just... check? They're logged in. You can see their profile right there. Why do you need OAuth?

This sounds naive. OAuth exists for good reasons — the website needs to confirm the user's identity to a third party, cryptographically, without the user just saying "trust me." But notice what OAuth actually does: GitHub runs some code, checks that you're logged in, and signs a token saying so. The important part is "checks that you're logged in." The signing is just infrastructure.

So: what if you could check that someone is logged in, in a way that produces a signed token, without GitHub's involvement? You would need two things: their browser session (to see what they see) and a trusted environment (so the check can't be faked). That's the whole paper.

If this works for GitHub, it works for everything. Airlines, pharmacies, county tax portals, Neopets. They're all just websites. None of them need to build an OAuth endpoint. None of them need to know they're participating.

1.1 Related Work

Identity Federation. OpenID Connect [19], OAuth 2.0 [9]. The assumption that IdPs cooperate.

Credential Bridging. CanDID [13], DelegaTEE [14], SoK: Data Sovereignty [5]. Burnt's Redacted File [**?**] uses a TDX-based CVM to prove the absence of emails from a specific sender. Our work: the degenerate case where the credential is "I'm logged in."

TLS Oracles and zkTLS. DECO [23], TLSNotary [20], Reclaim [17], Opacity [10], ORIGO [6], Janus [11]. TLS oracles prove content of a TLS session. We prove what a browser shows when logged in — a strictly easier problem.

Anonymous Credentials. ASC/U2SSO [1], zkcreds [18], Semaphore [8], World ID [22].

2 Three Problems and a Browser

The plan is: run a browser in a trusted environment, log in as the user, observe the page, sign an attestation. The website never cooperates and never knows. This requires solving three problems.

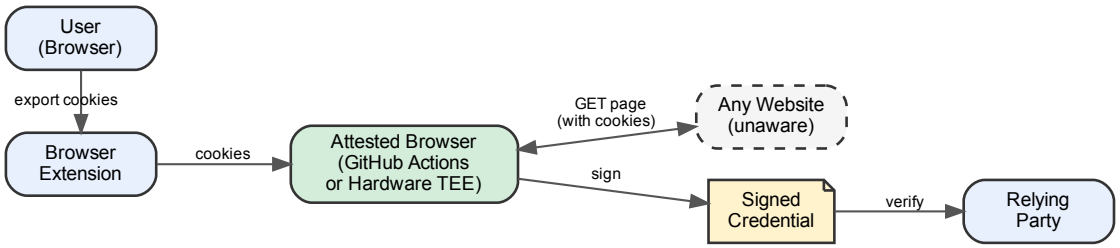

Figure 2: System architecture. The user exports cookies via a browser extension. The attested browser logs in, observes the page, and signs a credential. The website is unaware of its role.

2.1 Getting the Session

You can't log into someone's account without their session. A browser extension exports cookies for a target domain — the user clicks "prove," picks a site, and the extension packages their session for the attested environment. This is the only step that touches the user's machine.

2.2 Trusting the Browser

Whoever runs the browser controls the output. If I run it on my laptop, I can make it say anything. The browser needs to run somewhere I can't tamper with, and the output needs to be signed by that somewhere.

The obvious answer is a hardware TEE — SGX, SEV, Nitro. The actual answer, at least for credentials nobody is going to forge, is GitHub Actions. Each workflow runs in a fresh VM. Sigstore signs the artifact, binding it to the workflow YAML, commit SHA, and artifact hash [**?**, **?**]. The workflow file is pinned by commit hash, so auditors can verify the exact code that ran. This is attestation backed by Microsoft's reputation rather than Intel's silicon, but the threat model is spam bots and sockpuppets, not nation-states.

For high-stakes credentials (financial, medical), the same code runs in a hardware TEE via Dstack [**?**]. Only the trust root changes.

2.3 Handling Every Website

The browser can visit any URL, but "check if logged in" means different things on different sites. Each site needs an adapter. The key design decision: make adapters boring. An adapter defines a `poll()` function that returns a boolean, passes it to a shared observer that handles the loop, and that's it.

Listing 1: GitHub star adapter (actual code).

```
import { createObserver } from '../
    observer.js';

const repo = process.env.REPO;
const ghToken = process.env.GH_TOKEN;

async function poll() {
  const res = await fetch(
    `https://api.github.com/user/starred
        /${repo}`,
    { headers: {
        'Authorization': `Bearer ${
            ghToken}`,
        'Accept':
          'application/vnd.github.v3+
              json',
    }});
  return res.status === 204; // starred
}

createObserver({
  name: 'github-star',
  poll,
  description: `Watching ${repo}`,
}).run();
```

For sites we haven't seen before, an LLM generates the adapter: how to detect login state, what's extractable, what's interesting. The profile gets committed to the repo, pinned by hash, reused for the next user. Coverage grows with usage.

For interactive liveness — proving you control an account right now — the adapter can require observable actions: "like and unlike this tweet 10 times."

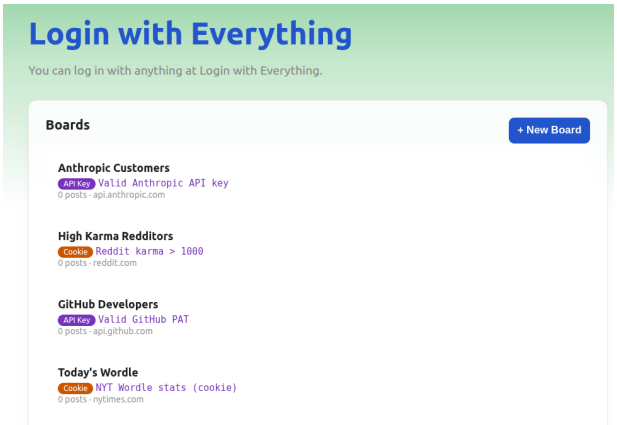

Figure 3: The forum application. Each board defines a login condition: an API key, a cookie-based credential, or a browser session proof. Users who pass the check can view and post.

3 Demo: Login with Everything

3.1 Forum Application

We built a forum where each board defines its own login condition. A board creator describes what credential is required — "Reddit karma above 1000," "valid Anthropic API key," "NYT Wordle stats via cookie" — and an LLM generates the verification plan: which site to check, how to detect login state, and what to extract.

Users who pass the check can view and post to the board. Users who don't pass see only that the board exists. Within each board, members are pseudonymous: a nullifier derived from their credential gives them a persistent identity ("same person as last time") without revealing who they are across boards.

The board itself is the relying party SDK in miniature. The creator never writes an OAuth integration or a login flow. They write a sentence describing who should be allowed in, and the infrastructure handles the rest.

4 What to Prove

Login with Neopets. If your account from 2003 still works, you predate the dead internet. Your starving Neopet has been waiting. No LLM would maintain a Neopets account. More generally, "login with pre-2022 anything" is a proof of humanity that gets stronger every year, costs nothing to verify, and is permanently unforgeable.

Login with your county tax portal. Jurisdictional KYC without showing your address. Your pharmacy login proves you take medication X for a drug interaction checker without revealing your name. Your electric utility knows your usage, your address, and whether you own the building. Carta doesn't need to build an OAuth endpoint for you to prove you hold shares. These sites have the data already — they just never agreed to be identity providers, and now they don't have to.

Login by logging out. To join this forum, prove you deactivated your Twitter account. Log back in, you're kicked. Prove you deleted your most popular post — the TEE confirms your top-performing content no longer exists. OAuth couldn't express these even if every site cooperated, because the credential is the *absence* of something.

Login with quorum. N of M people must authenticate within 60 seconds. The credential is "I have friends who will do something annoying simultaneously." Alternatively: two people must prove opposing positions, one logged into Fox News, the other MSNBC. Verified disagreement for structured debate.

5 Discussion

5.1 Nobody Needs a User Table

The attestation from Section 2 says "this person is logged into Neopets." That's useful, but it's also linkable — if you show the same attestation to two different forums, they can collude and figure out you're the same Neopets person.

The fix is standard: wrap the attestation in a zero-knowledge proof. Register a commitment $c = \mathcal{H}(\texttt{owner_id} \| s)$ on-chain, prove membership in the set of registered commitments, and output a domain-specific nullifier $\nu = \mathcal{H}(s \| domain)$. The nullifier is the same every time you visit the same forum (so they know you're "that Neopets person again") but different across forums (so no one can link you). This is not our contribution — it's Semaphore [8], it's zk-creds [18], it's the standard play. Our contribution is bootstrapping the master credential from a website that never agreed to participate.

What's new is what this does to the relying party. If the credential is self-proving, the app doesn't need to verify it against a database. Which means it doesn't need a database. Which means it doesn't need a user table, a signup flow, or a password reset page. The app says "show me proof you have AAdvantage Platinum" and either you can or you can't. There is nothing to store and nothing to delete.

GDPR compliance by construction: you can't leak what you never had.

The most interesting version is continuous. A group chat called `#currently-logged-into-neopets` means exactly what it says. The TEE re-checks periodically. You log out of Neopets, you get kicked. You log back in, you can rejoin. The channel name is the specification, the enforcement mechanism, and the documentation, all at once.

5.2 Credentials as Context

Everything so far uses a credential as a gate: you're in or you're out. But that's not actually how trust works. When someone reviews a product, you want to know how many they've bought. When someone calls a trade, you want to know their exposure. When someone files a bug report, you want to know if they actually run the software.

The credential doesn't decide whether you can speak. It decides how seriously to take you. This is a different thing entirely, and it might be the most useful application of the whole system — not because it's technically novel, but because it turns out the infrastructure for "prove you're logged into Neopets" is the same infrastructure for "weight this product review by verified purchase history." We did not plan this.

6 Security Analysis

Replay. Attestations are bound to timestamps and run IDs. Relying parties can require fresh runs.

Session theft. If an attacker steals cookies, they can produce fraudulent attestations. This is equivalent to the attacker being logged in — a pre-existing compromise the protocol does not worsen.

Covert channels. Twitter likes: $\sim$2 bits/second. Amazon cart: much higher bandwidth. The shopping cart as a covert communication channel is left to future work.

7 Conclusion

The entire concept of "login" is a historical accident. The only reason identity was required was that proving properties required first establishing identity and looking up the property in a database. Attested browser login inverts this: prove the property directly, skip the identity.

Acknowledgments

References

[1] Jayamine Alupotha, Mariarosaria Barbaraci, Ioannis Kaklamanis, Abhimanyu Rawat, Christian Cachin, and Fan Zhang. Anonymous self-credentials and their application to single-sign-on. Cryptology ePrint Archive, Paper 2025/618, 2025.

[2] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah Johnson, Ari Juels, Andrew Miller, and Dawn Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *IEEE EuroS&P*, 2019.

[3] Wikipedia contributors. Shave and a haircut. Wikipedia, The Free Encyclopedia, 2024. Accessed March 2026.

[4] Dan Elitzer and Peter Shoemaker. Crypto's AirTag moment. Nascent, `https://www.nascent.xyz/idea/cryptos-airtag-moment`, 2024.

[5] Jens Ernstberger, Jan Lauinger, Fatima Elsheimy, Liyi Zhou, Sebastian Steinhorst, Ran Canetti, Andrew Miller, Arthur Gervais, and Dawn Song. SoK: Data sovereignty. Cryptology ePrint Archive, Paper 2023/967, 2023.

[6] Jens Ernstberger, Jan Lauinger, Yinnan Wu, Arthur Gervais, and Sebastian Steinhorst. ORIGO: Proving provenance of sensitive data with constant communication. IACR ePrint 2024/447, 2024.

[7] Gitcoin. Gitcoin passport: Sybil resistance protocol. `https://passport.gitcoin.co`, 2023.

[8] Kobi Gurkan, Koh Wei Jie, and Barry White-Hat. Semaphore: Zero-knowledge signaling on Ethereum. `https://semaphore.pse.dev`, 2020.

[9] Dick Hardt. RFC 6749: The OAuth 2.0 authorization framework. IETF RFC 6749, 2012.

[10] Opacity Labs. Opacity network: Verified data network with zkTLS. `https://opacity.network`, 2024.

[11] Jan Lauinger, Jens Ernstberger, Andreas Finkenzeller, and Sebastian Steinhorst. Janus: Fast privacy-preserving data provenance for TLS. In *PETS*, 2025.

[12] Zhongtang Luo, Yanxue Jia, Yaobin Shen, and Aniket Kate. Proxying is enough: Security of proxying in TLS oracles and AEAD context unforgeability. IACR ePrint 2024/733, 2024.

[13] Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. CanDID: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability. In *IEEE S&P*, 2021.

[14] Sinisa Matetic, Moritz Schneider, Kari Kostiainen, Andrew Miller, Ari Juels, and Srdjan Capkun. DelegaTEE: Brokered delegation using trusted execution environments. In *USENIX Security*, 2018.

[15] Andrew Miller and Xinyuan Sun. Github as a trusted execution environment: Speedrunning permissionless credentials. In *Financial Cryptography and Data Security (FC)*, 2026.

[16] National Confectioners Association. Candy corn production statistics. NCA Industry Reports, 2023.

[17] Reclaim Protocol. Reclaim protocol: HTTPS proxies for verifiable web data. `https://reclaimprotocol.org`, 2024.

[18] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure. In *IEEE S&P*, 2023.

[19] Nat Sakimura, John Bradley, Michael Jones, Breno de Medeiros, and Chuck Mortimore. OpenID Connect Core 1.0. OpenID Foundation, 2014.

[20] TLSNotary. TLSNotary: A mechanism for independently audited TLS sessions. `https://tlsnotary.org`, 2023.

[21] E. Glen Weyl, Puja Ohlhaver, and Vitalik Buterin. Decentralized society: Finding Web3's soul. SSRN 4105763, 2022.

[22] World Foundation. World ID protocol: Privacy-preserving proof of personhood. `https://whitepaper.world.org`, 2024.

[23] Fan Zhang, Deepak Maram, Harjasleen Malvai, Steven Goldfeder, and Ari Juels. DECO: Liberating web data using decentralized oracles for TLS. In *ACM CCS*, 2020.